

Atividade: Do C ao Binário no PC

Na aula, o programa `g_a + g_b` imprimiu `10`, o `nm` mostrou `g_a` na `.data` e `g_b` na `.bss`, e restou uma resposta ainda a ser comprovada: quem posicionou o `10` em `g_a` e o `0` em `g_b`, antes mesmo do `main()` ser chamado, foram o **carregador** do sistema operacional e o **C runtime** — duas peças que você, em nenhum momento, escreveu. Nesta atividade, você verificará, experimento por experimento, cada afirmação dessa resposta diretamente no executável, e observará as outras duas peças em pleno funcionamento: a **biblioteca C** e o **sistema operacional**.

Tudo se passa integralmente no PC. Não há necessidade da placa; basta um terminal Linux (o WSL é suficiente) equipado com `gcc` e `strace`.

i Orientações

- **Pré-requisito:** um terminal Linux (o WSL é suficiente). Não é necessário dispor da placa, tampouco ter concluído o roteiro de Ambiente ou comparecido à aula: a seção Preparando o ambiente apresenta, passo a passo, tudo aquilo que precisa ser instalado e copiado.
- **Saídas esperadas:** cada experimento apresenta a saída que se espera obter. Endereços, tamanhos e números de linha podem variar discretamente conforme a versão do compilador e da distribuição utilizada; o que efetivamente importa é o que cada saída revela. Caso o seu sistema esteja configurado em português, as mensagens do compilador e do *linker* surgirão traduzidas. Para visualizá-las exatamente como neste texto, inicie o comando com `LANG=C` (por exemplo, `LANG=C gcc -nostdlib program.c -o nostd`).
- **Entrega:** um relatório, em formato PDF, contendo as respostas à seção Relatório, acompanhadas das saídas dos comandos que as justificam.
- **Respostas autorais:** elabore as suas próprias explicações a partir dos experimentos realizados. Respostas copiadas de fontes externas, ou geradas por inteligência artificial sem elaboração própria, não atingem o objetivo pedagógico da atividade.

Preparando o ambiente

Execute os passos a seguir **rigorosamente na ordem apresentada**, mesmo que julgue já tê-los realizado anteriormente. Os passos já concluídos não causam qualquer problema: o `apt` informa que o pacote já se encontra instalado, e os comandos marcados como condicionais explicam claramente quando devem ser ignorados.

1. Instale as ferramentas necessárias. O pacote `build-essential` fornece o `gcc` e o `make`; o `git` permite baixar o repositório da disciplina; o `strace` será utilizado na Parte 3:

```
foo@bar$ sudo apt update
foo@bar$ sudo apt install build-essential git strace
```

Confirme que os três respondem adequadamente:

```
foo@bar$ gcc --version
foo@bar$ git --version
foo@bar$ strace --version
```

Cada um dos comandos deve imprimir a respectiva versão instalada. Caso algum deles responda `command not found`, repita a instalação e leia atentamente a mensagem de erro fornecida pelo `apt`.

2. Crie a pasta de trabalho da disciplina (caso ela já exista, a opção `-p` não gera reclamação):

```
foo@bar$ mkdir -p ~/seml-workspace
foo@bar$ cd ~/seml-workspace
```

3. Baixe o repositório da disciplina. Verifique, antes de tudo, se ele já se encontra presente no seu sistema:

```
foo@bar$ ls ~/seml-workspace
```

Caso a lista **não** apresente a pasta `ufu-seml-src`, clone o repositório:

```
foo@bar$ git clone https://github.com/daniel-p-carvalho/ufu-seml-src.git
```

Caso a pasta **já exista**, é suficiente atualizá-la:

```
foo@bar$ cd ~/seml-workspace/ufu-seml-src
foo@bar$ git pull
foo@bar$ cd ~/seml-workspace
```

4. Copie os arquivos do laboratório. Caso você tenha comparecido à aula, a pasta `~/seml-workspace/lab-02` já existe (confirme com `ls ~/seml-workspace`), e pode ser reutilizada diretamente. Caso ela **não** exista, copie-a do repositório:

```
foo@bar$ cp -r ~/seml-workspace/ufu-seml-src/lab-02 ~/seml-workspace/lab-02
```

5. Entre na pasta e confira o seu conteúdo:

```
foo@bar$ cd ~/seml-workspace/lab-02
foo@bar$ ls
```

A listagem deve conter, entre outros arquivos, `program.c`, `embedded.c` e `cosine.c` (a ordem de exibição pode variar).

Todos os comandos desta atividade devem ser executados **a partir desta pasta**. Caso você encerre o terminal e retorne posteriormente, inicie sempre com `cd ~/seml-workspace/lab-02`.

6. Teste o ambiente: compile e execute o programa, exatamente como feito em aula:

```
foo@bar$ gcc program.c -o program
foo@bar$ ./program
g_a + g_b = 10
```

Caso tenha aparecido `g_a + g_b = 10`, o ambiente está devidamente preparado.

Antes de começar: lendo as linhas de comando

Diversos comandos empregados nesta atividade encadeiam múltiplos programas entre si. Cinco recursos do *shell*, especificamente, aparecem com frequência:

- **| (pipe)** conecta a saída de um programa diretamente à entrada do programa seguinte. Em `nm program | grep main`, o `grep` recebe a lista de símbolos produzida pelo `nm` e exibe apenas as linhas que contêm `main`.
- **grep -E** aceita expressões regulares estendidas. Em `grep -E " (g_a|g_b)$"`, a barra vertical situada entre os parênteses significa “ou”, o `$` significa “fim de linha”, e o espaço inicial constitui um espaço literal: o padrão casa, portanto, com as linhas que **terminam** em `g_a` ou `g_b`, e não com `g_abc`. A opção `-i` desconsidera a diferença entre maiúsculas e minúsculas, ao passo que `-A 15` exibe também as 15 linhas subsequentes a cada ocorrência encontrada.
- **2>&1** funde a saída de erro à saída padrão. Todo programa possui, na realidade, duas saídas distintas: a padrão (descriptor 1) e a de erro (descriptor 2). O *pipe*, por si só, transporta unicamente a saída padrão; o `gcc -v`, contudo, escreve na saída de erro. Na ausência do `2>&1`, o `grep` nada receberia.
- **;** executa um comando após o outro, incondicionalmente. `$?`, por sua vez, é um parâmetro especial do *shell*, responsável por armazenar o **código de saída** do último comando executado: o valor `0` indica sucesso, e qualquer outro valor indica falha. A expressão `./program; echo $?` executa o programa e, em seguida, exibe o código com que ele foi finalizado.
- **tr ' ' '\n'** substitui cada espaço por uma quebra de linha, recurso útil para fragmentar uma linha de comando excessivamente longa em uma palavra por linha, permitindo, então, filtrá-la com o `grep`.

Para saber mais: expressões regulares

O `grep -E` empregado nesta atividade utiliza apenas três recursos de expressões regulares, mas esses recursos valem amplamente o investimento de aprendizado: eles reaparecem no `grep`, no `sed`, nos editores de código e em praticamente toda linguagem de programação.

- **Expressões Regulares — Guia de Consulta Rápida**, de Aurelio Marinho Jargas (Novatec). Em português, gratuito e completo na internet. Curto e direto, com tabelas de todos os metacaracteres. **Recomenda-se iniciar por aqui.**
- **Manual do GNU grep**, Seção 3.6 (*Basic vs Extended Regular Expressions*). Explica por que razão o `grep -E` aceita `|`, `( e )` sem barra invertida, ao passo que o `grep` simples não o faz. No terminal, `man 7 regex` apresenta o mesmo conteúdo.
- **Testadores interativos**, tais como o `regex101` e o `RegExr`, evidenciam o que determinado padrão casa à medida que se digita. Atenção: esses testadores empregam, por padrão, uma

sintaxe mais rica (PCRE ou JavaScript), com recursos como `\d`, que o `grep -E` não reconhece. Um padrão funcional nesses testadores pode, portanto, falhar no terminal.

Parte 1 — As etapas, uma a uma

Confirme, antes de prosseguir, que você se encontra na pasta do laboratório (`cd ~/seml-workspace/lab-02`).

O **pré-processamento** é solicitado por meio da opção `-E`:

```
foo@bar$ gcc -E program.c -o program.i
foo@bar$ wc -l program.c program.i
  44 program.c
 860 program.i
 904 total
foo@bar$ tail -n 20 program.i
```

O arquivo `program.i` possui centenas de linhas adicionais em relação a `program.c`: trata-se do conteúdo do `stdio.h` e dos demais arquivos por ele incluídos. Ao final, encontra-se o nosso próprio código, já despido dos comentários e com `INITIAL_VALUE` devidamente substituído por `10`. Localize a declaração de `printf` com `grep -n printf program.i`: trata-se apenas de uma declaração, sem o código propriamente dito da função.

A **compilação** para *assembly* é solicitada por meio da opção `-S`:

```
foo@bar$ gcc -S program.c -o program.s
foo@bar$ less program.s
```

Localize os rótulos `g_a`: e `g_b`:. Precedendo `g_a`, encontra-se a diretiva `.data`, com o valor `10` registrado imediatamente abaixo (`.long 10`); precedendo `g_b`, encontra-se `.bss`, e, em vez de um valor propriamente dito, apenas a reserva de 4 bytes (`.zero 4`). O compilador separou, portanto, as duas variáveis em seções distintas: `g_a` necessita carregar o seu valor inicial, ao passo que `g_b` requer apenas espaço reservado.

A **montagem** é solicitada por meio da opção `-c`. Tendo já observado, em aula, o `nm` do objeto, verifique agora o que o `file` e o `objdump` revelam a respeito dele:

```
foo@bar$ gcc -c program.c -o program.o
foo@bar$ file program.o
program.o: ELF 64-bit LSB relocatable, x86-64, version 1 (SYSV), not stripped
foo@bar$ objdump -d program.o
```

O objeto é **relocável** (*relocatable*): ainda não possui endereços definitivos. No `objdump -d`, observe que a chamada a `printf`, bem como os acessos a `g_a` e `g_b`, apresentam deslocamentos zerados: correspondem, precisamente, às relocações que o *linker* será responsável por preencher.

A **ligação** corresponde àquilo que o `gcc` executa quando recebe objetos, em vez de arquivos-fonte:

```

foo@bar$ gcc program.o -o program
foo@bar$ file program
program: ELF 64-bit LSB pie executable, x86-64, ..., dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2, ...
foo@bar$ nm program | grep -E " (g_a|g_b|add|main)$"
0000000000001149 T add
0000000000004010 D g_a
0000000000004018 B g_b
0000000000001161 T main

```

Neste momento, o `file` indica *executable*, e os símbolos passam a apresentar endereços definitivos. Para observar a cadeia completa de uma só vez, solicite ao `gcc` que preserve os arquivos intermediários (`-save-temps`) e exiba os programas que efetivamente executa (`-v`):

```

foo@bar$ gcc -save-temps -v program.c -o program 2>&1 | grep -E "cc1|as |collect2"
/usr/libexec/gcc/x86_64-linux-gnu/15/cc1 -E -quiet -v ... program.c ...
/usr/libexec/gcc/x86_64-linux-gnu/15/cc1 -fpreprocessed program.i ...
/usr/bin/x86_64-linux-gnu-as -v --64 -o program.o program.s
/usr/libexec/gcc/x86_64-linux-gnu/15/collect2 ...

```

O `cc1` aparece em duas ocasiões distintas: primeiramente com `-E`, executando o pré-processamento, e, posteriormente, compilando o `program.i` já pré-processado. Em seguida, surgem o `as` (o *assembler*) e o `collect2`, responsável por invocar o *linker* `ld`. O `gcc`, em si mesmo, não compila absolutamente nada: trata-se de um **driver**, cuja função é invocar cada programa da cadeia na ordem apropriada.

O tamanho de cada seção é revelado pelo `size`:

```

foo@bar$ size program.o program
   text    data     bss     dec      hex filename
   220         4         4     228      e4 program.o
  1474       604        12    2090     82a program

```

No objeto, a `.data` possui 4 bytes (`g_a`) e a `.bss` possui igualmente 4 (`g_b`). No executável, ambas cresceram: o *runtime* e a biblioteca acrescentaram as suas próprias variáveis. Substitua `int g_b;` por `int g_b = 5;`, recompile e execute novamente o `size`: `g_b` passa a ser contabilizada na `data` (608) e deixa de figurar na `bss` (8), uma vez que agora também possui um valor inicial a preservar. Restaure, então, a declaração original `int g_b;`.

Altere a localização de `g_b`. Copie o arquivo `program.c` para `local.c` e, nessa cópia, elimine a declaração global de `g_b`, declarando, no interior do `main`, uma variável local `int b;`, empregada em substituição a `g_b` na chamada a `add`. Compile com os avisos habilitados:

```
foo@bar$ gcc -Wall local.c -o local
local.c:...: warning: 'b' is used uninitialized [-Wuninitialized]
foo@bar$ ./local
g_a + b = 10
foo@bar$ nm local | grep -E " (g_a|b)$"
0000000000004010 D g_a
```

O compilador emite um aviso, informando que `b` é utilizada sem receber valor: variáveis locais **não** são zeradas pela linguagem. Além disso, o símbolo desapareceu do `nm`, uma vez que não reside em seção alguma: é criado na **pilha** no momento em que o `main()` tem início. Ainda assim, o programa imprimiu 10 — o que não constitui garantia da linguagem C, mas simples coincidência, decorrente do fato de o Linux entregar a pilha de um processo recém-criado já zerada. Valor **indeterminado**, portanto, não equivale a valor aleatório. Substitua, agora, `int b;` por `static int b;`:

```
foo@bar$ gcc -Wall static.c -o static
foo@bar$ nm static | grep " b"
0000000000004018 b b.0
```

O aviso desaparece, e `b` retorna à `.bss`, agora com zero garantido. O qualificador `static`, quando aplicado a uma variável local, altera o seu **tempo de vida** (ela passa a existir durante toda a execução do programa), mas não altera o seu **escopo** (permanece visível exclusivamente no interior do `main`). Observe a letra minúscula `b`, que indica tratar-se de um símbolo **local** ao arquivo, bem como o sufixo `.0`, acrescentado pelo compilador ao nome, com o propósito de distingui-lo de outras eventuais variáveis `b` estáticas.

Parte 2 — Quem fez `g_a = 10` e `g_b = 0`?

Quem, de fato, dá início ao programa? Consulte o executável a respeito de qual é o seu ponto de entrada:

```
foo@bar$ readelf -h program | grep -i -E "entry|entrada"
  Entry point address:                0x1060
foo@bar$ nm program | grep -E " (_start|main)$"
0000000000001161 T main
0000000000001060 T _start
foo@bar$ objdump -d program | grep -A 15 "<_start>:"
```

O ponto de entrada (*entry point*) não corresponde ao `main`, mas sim ao `_start`. Na desmontagem, observa-se que ele organiza a pilha, carrega o endereço de `main` em um registrador (a linha contendo `lea ... <main>`) e, em seguida, invoca `__libc_start_main`, pertencente à biblioteca C, responsável por inicializar a biblioteca e a entrada e saída padrão, preparar `argc` e `argv`, chamar o `main()` e, uma vez que este retorne, transmitir o valor devolvido ao sistema operacional. É exatamente por essa razão que o `return 0;` se converte no código de saída do programa:

```
foo@bar$ ./program; echo $?  
g_a + g_b = 10  
0
```

Substitua `return 0;` por `return 42;`, recompile e verifique o novo código de saída. Restaure, posteriormente, o `return 0;`.

E quanto a `g_a` e `g_b`? No momento em que você executa `./program`, antes mesmo de o `_start` ser alcançado, o **carregador** do sistema operacional lê o executável, copia a seção `.data` para a memória — trazendo consigo o 10 de `g_a` — e reserva a `.bss` já devidamente zerada. Os valores encontravam-se armazenados no arquivo; o sistema operacional os transferiu para a memória; o *runtime*, por sua vez, preparou o restante.

Sem o *runtime*. O `_start` encontra-se em um dos arquivos objeto do *runtime*, que o `gcc` entrega ao *linker* sem que isso lhe seja solicitado explicitamente. Observe-os na própria linha de comando do *linker*:

```
foo@bar$ gcc -v program.c -o program 2>&1 | grep collect2 | tr ' ' '\n' | grep crt  
/usr/lib/gcc/x86_64-linux-gnu/15/../../../../x86_64-linux-gnu/Scrt1.o  
/usr/lib/gcc/x86_64-linux-gnu/15/../../../../x86_64-linux-gnu/crti.o  
/usr/lib/gcc/x86_64-linux-gnu/15/crtbeginS.o  
/usr/lib/gcc/x86_64-linux-gnu/15/crtendS.o  
/usr/lib/gcc/x86_64-linux-gnu/15/../../../../x86_64-linux-gnu/crtn.o
```

Os arquivos `Scrt1.o` (no qual reside o `_start`), `crti.o`, `crtbeginS.o`, `crtendS.o` e `crtn.o` constituem, em conjunto, o *runtime*. Ligue, agora, o programa sem que eles estejam presentes:

```
foo@bar$ gcc -nostartfiles program.c -o no-crt  
/usr/bin/ld: warning: cannot find entry symbol _start; defaulting to  
0000000000001030  
foo@bar$ ./no-crt  
Segmentation fault (core dumped)  
foo@bar$ echo $?  
139
```

O *linker* adverte que não localizou `_start` e opta por um endereço arbitrário como ponto de entrada. O programa é encerrado com **falha de segmentação**: ninguém prepara adequadamente o ambiente, ninguém invoca o `main()` da maneira correta e, quando este retorna, não há para onde retomar a execução. O código de saída 139 corresponde a $128 + 11$, sendo 11 o número do sinal `SIGSEGV`, empregado pelo *kernel* para encerrar um processo que acessa memória inválida. Guarde cuidadosamente este experimento: em um microcontrolador não existe sistema operacional capaz de carregar a `.data`, nem *runtime* do PC que sirva a esse propósito, e quem desempenhará esse papel será um código que nós mesmos escreveremos, o `startup.c`.

Parte 3 — Onde está o `printf`, e como o texto chega ao terminal?

Cabeçalho ou biblioteca? Três experimentos distintos permitem separar essas duas coisas. Em primeiro lugar, copie o `program.c` para `noinc.c`, elimine nele a linha `#include <stdio.h>` e compile:

```
foo@bar$ gcc noinc.c -o noinc
noinc.c: In function 'main':
noinc.c:...: error: implicit declaration of function 'printf' [-Wimplicit-
function-declaration]
noinc.c:1:1: note: include '<stdio.h>' or provide a declaration of 'printf'
```

O erro é originado pelo **compilador**: na ausência do cabeçalho, ele desconhece o que vem a ser `printf`. Agora, no lugar do `#include`, escreva você mesmo a declaração que o cabeçalho, de outro modo, forneceria:

```
int printf(const char *format, ...);
```

Compile e execute o programa: ele funciona exatamente como antes. Tudo aquilo que o `stdio.h` fornecia ao nosso programa reduzia-se, portanto, a essa única linha. Por fim, retorne ao `program.c`, mantendo o `#include`, e solicite ao `gcc` que **não** acrescente a biblioteca C (opção `-nostdlib`):

```
foo@bar$ gcc -nostdlib program.c -o nostd
/usr/bin/ld: warning: cannot find entry symbol _start; defaulting to
0000000000001030
/usr/bin/ld: /tmp/cc...o: in function `main':
program.c:(.text+0x49): undefined reference to `printf'
collect2: error: ld returned 1 exit status
```

Desta vez, o erro é originado pelo **linker**: o cabeçalho encontra-se presente, mas o código da função, não. O aviso relativo ao `_start` reaparece, uma vez que, na ausência da biblioteca, tampouco há *runtime*.

Onde se encontra o código do `printf`? O `ldd` enumera as bibliotecas compartilhadas das quais um executável depende:

```
foo@bar$ ldd program
linux-vdso.so.1 (0x...)
libc.so.6 => /usr/lib/x86_64-linux-gnu/libc.so.6 (0x...)
/lib64/ld-linux-x86-64.so.2 (0x...)
```

A linha `libc.so.6` corresponde à biblioteca C do Linux (a *glibc*), de onde efetivamente provém o `printf`: o `program` foi ligado **dinamicamente**, e a biblioteca somente é carregada a partir do disco no momento em que o programa é executado. A última linha refere-se ao próprio carregador dinâmico. Gere, agora, uma versão ligada **estaticamente**, e proceda à comparação:

```
foo@bar$ gcc -static program.c -o program-static
foo@bar$ ldd program-static
        not a dynamic executable
foo@bar$ ls -l program program-static
foo@bar$ size program program-static
      text      data       bss       dec       hex filename
      1474        604         12      2090        82a  program
 699491  22776   22560  744827   b5d7b  program-static
```

O executável estático é aproximadamente **50 vezes maior** (no `ls -l`, algo em torno de 16 KB contra 800 KB): um único `printf` carrega consigo formatação de números, *buffers*, suporte a idiomas e tratamento de erros. Em um microcontrolador, não havendo sistema operacional capaz de carregar bibliotecas, a ligação é invariavelmente estática, e esse custo adicional pesa consideravelmente sobre uma Flash de 512 KB.

De que forma o texto chega ao terminal? Um programa comum não possui permissão para acessar diretamente o hardware. Para escrever no terminal, ele efetua, portanto, uma **chamada de sistema** (*system call*, ou *syscall*): um pedido dirigido ao *kernel*. O `strace` executa um programa e exibe, uma por linha, a totalidade das *syscalls* por ele realizadas. Cada linha assume a forma `nome(argumentos) = valor de retorno`. Execute:

```
foo@bar$ strace ./program
```

A saída compreende cerca de 35 linhas. **Praticamente todas elas pertencem ao carregador**, encarregado de preparar o processo antes do `main()`: `execve` (o *kernel* dá início ao programa), `brk` (prepara o *heap*), `openat` e `mmap` (abrem a `libc.so.6` e a mapeiam na memória), `mprotect`, entre outras. Ignore-as por ora. As linhas efetivamente correspondentes ao nosso código são as últimas:

```
write(1, "g_a + g_b = 10\n", 15g_a + g_b = 10
)      = 15
exit_group(0)                                = ?
+++ exited with 0 +++
```

O texto do programa aparece entrelaçado com a linha do `strace`, uma vez que ambos escrevem no mesmo terminal. Interprete a saída da seguinte forma: `write(1, "g_a + g_b = 10\n", 15)` = 15 solicita ao *kernel* que escreva 15 bytes no descritor 1, correspondente à saída padrão, e o *kernel* responde informando que os 15 bytes foram, de fato, escritos. Todo o trabalho executado pelo `printf` — interpretar o `%d`, converter o número em texto, montar a *string* resultante — foi realizado pela própria biblioteca, no interior do programa; somente ao final ela efetuou a chamada a `write`. E o `return 0`; , por sua vez, converteu-se em `exit_group(0)`. Para um aprofundamento adicional, consulte `man 2 write` e `man 2 syscalls`.

Compare esse resultado com a versão estática e com o resumo produzido pela opção `-c`, responsável por contabilizar as chamadas de cada tipo:

```
foo@bar$ strace ./program-static
foo@bar$ strace -c ./program
```

A versão estática realiza aproximadamente 15 *syscalls*, contra cerca de 35 na versão dinâmica: na ausência de bibliotecas a serem abertas e mapeadas, resta pouco além do `write` e do `exit_group`.

Figura: as camadas de um `printf` no Linux — o programa chama `printf` (biblioteca C, formatação e buffer), que chama a *syscall* `write`, que entra no kernel, que aciona o driver do terminal; ao lado, a mesma pilha em um microcontrolador bare-metal, onde as camadas do kernel e do driver não existem.

Relatório

Responda com as suas próprias palavras, apresentando as saídas dos comandos que justificam cada resposta:

1. Qual foi a previsão formulada em aula, antes da execução do programa, a respeito de sua saída? Explique o resultado correto e responda, fundamentando-se nos experimentos realizados: quem foi responsável por `g_a = 10` e `g_b = 0`? Indique em que seção ficou posicionada cada variável, inclusive na versão com `int g_b = 5`; e descreva o que ocorreu com `b` nas versões `local` e `static local`.
2. Explique, passo a passo, por que razão o `no-crt` é encerrado com falha de segmentação. O que o `_start` realiza, e que o nosso programa deixou de possuir?
3. Compare as saídas do `strace` obtidas com `program` e com `program-static`. Quantas *syscalls* cada um efetuou, e a que se deve essa diferença? Quais delas correspondem diretamente a linhas do `program.c`?
4. Em aula, a ligação do `program.c` para o Cortex-M4 fracassou, solicitando `_write`, `_read`, `_sbrk`, `_exit` e outras funções. Compare essa lista com as *syscalls* observadas no `strace`. O que elas possuem em comum, e por que motivo a `newlib` não pode implementá-las de forma autônoma?
5. Para cada uma das funções `_write`, `_sbrk` e `_exit`, pesquise qual deveria ser o seu comportamento e proponha o que ela poderia realizar em um microcontrolador desprovido de sistema operacional. (Pista: no livro do Yiu, a Seção 18.2.4, p. 590, apresenta um `_write` escrito para o GCC, e a Seção 18.4, *Re-targeting to peripherals*, pp. 595–604, demonstra como redirecionar a saída do `printf` para um periférico.)
6. Escreva uma função com **seis** argumentos inteiros que retorne a soma de todos eles. Compile-a com `arm-none-eabi-gcc -S -O1 -mcpu=cortex-m4 -mthumb` e explique, com base no *assembly* gerado, onde a função localiza o quinto e o sexto argumentos.
7. O arquivo `cosine.c`, presente no `lab-02`, emprega a função `cos`, pertencente à biblioteca matemática. Compile-o primeiramente com `gcc cosine.c -o cosine` e, em seguida, com `gcc cosine.c -o cosine -lm`. Explique a diferença entre os dois resultados obtidos, considerando que o `#include <math.h>` está presente em ambos os casos. Qual etapa falhou na primeira tentativa, e o que a opção `-lm` efetivamente requisita?

Para ir além (opcional). A versão local de `b` imprimiu 10 por mera coincidência. Observe o “lixo” de verdade: escreva uma função `dirty()`, que apenas declara uma variável local `int x = 1234`; e

uma função `add_with_local()`, que declara uma local `int b`; e retorna `g_a + b`. Chame `dirty()` e, imediatamente depois, imprima o valor de `add_with_local()`. Compile sem otimização. O que foi efetivamente impresso, e por qual razão? (Pista: as duas funções utilizam a mesma porção da pilha, uma sucessivamente após a outra.)

! Entrega e Controle de Versão

A entrega consiste no envio das respostas do Relatório — redigidas em um arquivo `LEIAME.md` dentro da pasta `lab-02` — e de todos os arquivos-fonte da atividade (`.c` e `.s`) para o seu repositório `seml-entregas`.

Para realizar esse envio, consolidaremos a rotina de controle de versão:

1. Verifique o estado atual: execute `git status`. Observe que o Git listará os arquivos-fonte novos e também os dezenas de artefatos gerados pelo `gcc` no PC, incluindo os executáveis sem extensão (como `program` e `program-static`). O `.gitignore` configurado na semana anterior ignorava artefatos ARM (`.elf`, `.bin`), mas não prevê executáveis nativos de Linux sem extensão.

2. Atualize o `.gitignore`: não é boa prática poluir o repositório com binários compilados (eles mudam a cada compilação e incham o histórico desnecessariamente). Abra o arquivo `~/seml-workspace/.gitignore` e adicione explicitamente os nomes dos executáveis gerados nesta prática:

```
# Executáveis nativos (lab-02)
program
program-static
local
static
noinc
nostd
no-crt
cosine
```

Verifique novamente com `git status`. Os binários devem desaparecer da lista de *untracked files*.

3. Prepare e registre (Boas mensagens de commit):

```
foo@bar$ git add .
foo@bar$ git commit -m "Seu texto aqui"
```

Uma mensagem de commit ruim: “lab 2” ou “arquivos novos”. *Uma mensagem de commit boa:* “Adiciona exemplos em C e respostas do relatório sobre runtime e syscalls”. O formato ideal utiliza o verbo no imperativo presente, descrevendo o que o *commit* faz com o repositório.

4. Envie: execute `git push` para sincronizar as mudanças com o GitHub.

Referências

- [1] Using the GNU Compiler Collection (GCC) — Overall Options — opções `-E`, `-S`, `-c`, `-save-temps`, `-v`, `-nostartfiles`, `-static`.
- [2] GNU Binary Utilities — `nm`, `objdump`, `size`, `readelf`.
- [3] Manual do Linux: `man 2 write`, `man 2 syscalls`, `man 1 strace`, `man 8 ld.so`, `man 7 signal`.
- [4] Kernighan, B. W.; Ritchie, D. M. *The C Programming Language*, 2ª edição. Prentice Hall, 1988. Seções 4.4 a 4.9 (escopo, variáveis estáticas e inicialização).